Experiment 6

Aim: To implement Text Processing Models

Code:

Text preprocessing models

```
import pandas as pd
from sklearn.datasets import fetch_20newsgroups import re
import nltk
from nltk.corpus import stopwords
from nltk.tokenize import word_tokenize

# Download NLTK resources nltk.download('punkt',
quiet=True) nltk.download('stopwords', quiet=True)

# Load the training data
newsgroups_train = fetch_20newsgroups(subset='train', remove=('headers', 'footers', 'quotes'))

# Create a DataFrame
df = pd.DataFrame({'data': newsgroups_train.data, 'target': newsgroups_train.target})

# Display the first few rows
display(df.head())
```


```
{"summary":"{\n  \"name\": \"display(df\",\n  \"rows\": 5,\n
\"fields\": [\n                    {\n                \"column\": \"data\",\n
\"properties\": {\n                                \"dtype\": \"string\",\n
\"num_unique_values\": 5,\n                                \"samples\": [\n                                \"A fair
number of brave souls who upgraded their SI clock oscillator have\\ nshared their experiences for this
poll. Please send a brief message detailing\\nyour experiences with the procedure. Top speed attained,
CPU rated speed,\\nadd on cards and adapters, heat sinks, hour of usage per day, floppy
disk\\nfunctionality with 800 and 1.4 m floppies are especially requested.\\n\\nI will be summarizing in
the next two days, so please add to the network\\nknowledge base if you have done the clock upgrade
and haven't answered this\\npoll. Thanks.\",\n
\"From article <C5owCB.n3p@world.std.com>, by tombaker@world.std.com (Tom A
Baker):\\n\\n\\nMy understanding is that the 'expected errors' are basically\\nknown bugs in the warning
system software - things are checked\\nthat don't have the right values in yet because they aren't\\nset till
after launch, and suchlike. Rather than fix the code\\nand possibly introduce new bugs, they just tell the
crew\\n'ok, if you see a warning no. 213 before liftoff, ignore it'.\",\n
\"well folks, my mac plus finally gave up the ghost this weekend after\\nstarting life as a 512k way back
in 1985.   sooo, i'm in the market for a\\nnew machine a bit sooner than i intended to be...\\n\\ ni'm
looking into picking up a powerbook 160 or maybe 180 and have a
```

bunch\\nof questions that (hopefully) somebody can answer:\\n\\n* does anybody know any dirt on when the next round of powerbook\\ nintroductions are expected? i'd heard the 185c was supposed to make an\\nappearence \\\"this summer\\\" but haven't heard anymore on it - and since i\\ndon't have access to macleak, i was wondering if anybody out there had\\nmore info...\\n\\n* has anybody heard rumors about price drops to the powerbook line like the\\nones the duo's just went through recently?\\n\\n* what's the impression of the display on the 180? i could probably swing\\na 180 if i got the 80Mb disk rather than the 120, but i don't really have\\na feel for how much
\\\"better\\\" the display is (yea, it looks great in the\\nstore, but is that all \\\"wow\\\" or is it really that good?). could i solicit\\nsome opinions of people who use the 160 and 180 day-to-day on if its worth\\ntaking the disk size and money hit to get the active display? (i realize\\nthis is a real subjective question, but i've only played around with the\\nmachines in a computer store breifly and figured the opinions of somebody\\nwho actually uses the machine daily might prove helpful).\\n\\n* how well does hellcats perform? ;)\\n\\ nthanks a bunch in advance for any info - if you could email, i'll post a\\nsummary (news reading time is at a premium with finals just around the\\ncorner... :( )\\n--\\nTom Willis \\\\ twillis@ecn.purdue.edu \\\\ Purdue Electrical Engineering\"\n],\n \"semantic_type\": \"\",\n \"description\": \"\"\n}\n },\n {\n \"column\": \"target\",\n \"properties\": {\n \"dtype\": \"number\",\n \"std\": 4,\n \"min\": 1,\n \"max\": 14,\n \"num_unique_values\": 4,\n \"samples\": [\n 4,\n 14,\n 7\n],\n \"semantic_type\": \"\",\n \"description\": \"\"\n }\n }\n]\n}","type":"dataframe"}

```
# Download punkt_tab resource
nltk.download('punkt_tab', quiet=True)

def preprocess_text(text):  # Convert to
    lowercase text = text.lower()
    # Remove punctuation and special characters
    text = re.sub(r'[^a-z\s]', '', text)
    # Tokenize
    tokens = word_tokenize(text)
    # Remove stop words
    stop_words = set(stopwords.words('english'))
    tokens = [word for word in tokens if word not in stop_words]
    # Join tokens back into a string
    return ' '.join(tokens)

# Apply preprocessing
df['processed_text'] = df['data'].apply(preprocess_text)
```

```
# Display original and processed text
display(df[['data', 'processed_text']].head())
```


```
{"summary":"{\n  \"name\": \"display(df[['data', 'processed_text']]\",\n  \"rows\": 5,\n
\"fields\": [\n                                                                {\n
\"column\": \"data\",\n                        \"properties\": {\n                        \"dtype\":
\"string\",\n            \"num_unique_values\": 5,\n                                    \"samples\": [\n
                    \"A fair number of brave souls who upgraded their SI clock oscillator have\\nshared
their experiences for this poll. Please send a brief message detailing\\nyour experiences with the
procedure. Top speed attained, CPU rated speed,\\nadd on cards and adapters, heat sinks, hour of usage
per day, floppy disk\\nfunctionality with 800 and
1.4 m floppies are especially requested.\\n\\nI will be summarizing in the next two days, so please add to
the network\\nknowledge base if you have done the clock upgrade and haven't answered this\\npoll.
Thanks.\",\n                        \"From article <C5owCB.n3p@world.std.com>, by
tombaker@world.std.com (Tom A Baker):\\n\\n\\nMy understanding is that the 'expected errors' are
basically\\nknown bugs in the warning system software - things are checked\\nthat don't have the right
values in yet because they aren't\\nset till after launch, and suchlike. Rather than fix the code\\nand
possibly introduce new bugs, they just tell the crew\\n'ok, if you see a warning no. 213 before liftoff,
ignore it'.\",\n                    \"well folks, my mac plus finally gave up the ghost this weekend
after\\nstarting life as a 512k way back in 1985.  sooo, i'm in the market for a\\nnew machine a bit
sooner than i intended to be...\\n\\ni'm looking into picking up a powerbook 160 or maybe 180 and have
a bunch\\nof questions that (hopefully) somebody can answer:\\n\\n* does anybody know any dirt on
when the next round of powerbook\\nintroductions are expected?  i'd heard the 185c was supposed to
make an\\nappearence \\\"this summer\\\" but haven't heard anymore on it - and since i\\ndon't have
access to macleak, i was wondering if anybody out there had\\nmore info...\\n\\n* has anybody heard
rumors about price drops to the powerbook line like the\\nones the duo's just went through
recently?\\n\\n* what's the impression of the display on the 180?  i could probably swing\\na 180 if i got
the 80Mb disk rather than the 120, but i don't really have\\na feel for how much \\\"better\\\" the display
is (yea, it looks great in the\\ nstore, but is that all \\\"wow\\\" or is it really that good?).
could i solicit\\nsome opinions of people who use the 160 and 180 day-
to-day on if its worth\\ntaking the disk size and money hit to get the active display?  (i realize\\nthis is a
real subjective question, but i've only played around with the\\nmachines in a computer store breifly and
figured the opinions of somebody\\nwho actually uses the machine daily might prove helpful).\\n\\n*
how well does hellcats perform?  ;)\\n\\nthanks a bunch in advance for any info - if you could email, i'll
post a\\nsummary (news reading time is at a premium with finals just around the\\ncorner... :( )\\n--
\\nTom Willis  \\\\ twillis@ecn.purdue.edu            \\\\     Purdue Electrical Engineering\"\n
],\n                \"semantic_type\": \"\",\n                                \"description\": \"\"\n
}\n          },\n            {\n              \"column\": \"processed_text\",\n
```

\"properties\": {\n \"dtype\": \"string\",\n \"num_unique_values\": 5,\n \"samples\": [\n \"fair number brave souls upgraded si clock oscillator shared experiences poll please send brief message detailing experiences procedure top speed attained cpu rated speed add cards adapters heat sinks hour usage per day floppy disk functionality floppies especially requested summarizing next two days please add network knowledge base done clock upgrade havent answered poll thanks\",\n \"article cowcbnpworldstdcom tombakerworldstdcom tom baker understanding expected errors basically known bugs warning system software things checked dont right values yet arent set till launch suchlike rather fix code possibly introduce new bugs tell crew ok see warning liftoff ignore\",\n \"well folks mac plus finally gave ghost weekend starting life k way back sooo im market new machine bit sooner intended im looking picking powerbook maybe bunch questions hopefully somebody answer anybody know dirt next round powerbook introductions expected id heard c supposed make appearence summer havent heard anymore since dont access macleak wondering anybody info anybody heard rumors price drops powerbook line like ones duos went recently whats impression display could probably swing got mb disk rather dont really feel much better display yea looks great store wow really good could solicit opinions people use daytoday worth taking disk size money hit get active display realize real subjective question ive played around machines computer store breifly figured opinions somebody actually uses machine daily might prove helpful well hellcats perform thanks bunch advance info could email ill post summary news reading time premium finals around corner tom willis twillisecnpurdueedu purdue electrical engineering\"\n],\n \"semantic_type\": \"\",\n \"description\": \"\"\n }\n }\n]\n}","type":"dataframe"}

```
from sklearn.feature_extraction.text import CountVectorizer

# Create bigrams
bigram_vectorizer = CountVectorizer(ngram_range=(2, 2)) bigram_matrix =
bigram_vectorizer.fit_transform(df['processed_text']) bigram_feature_names =
bigram_vectorizer.get_feature_names_out() print("Bigram features (first 20):")
print(bigram_feature_names[:20])

# Create trigrams
trigram_vectorizer = CountVectorizer(ngram_range=(3, 3)) trigram_matrix =
trigram_vectorizer.fit_transform(df['processed_text']) trigram_feature_names =
trigram_vectorizer.get_feature_names_out() print("\nTrigram features (first 20):")
print(trigram_feature_names[:20])
```

```
Bigram features (first 20):
['aa aaa' 'aa albany' 'aa anyway' 'aa apr' 'aa astronomy' 'aa atlanta'
```

```
 'aa batteries' 'aa bb' 'aa cida' 'aa early' 'aa fax' 'aa fire' 'aa four'
 'aa fri' 'aa however' 'aa improve' 'aa limited' 'aa mon' 'aa moved'
 'aa nd']

Trigram features (first 20):
['aa aaa improve' 'aa aaa majors' 'aa albany hopefully' 'aa anyway bob'
 'aa apr edt' 'aa astronomy astrophysics' 'aa atlanta well'
 'aa batteries also' 'aa batteries provided' 'aa batteries track'
 'aa bb cc' 'aa cida ajteel' 'aa cida ajteeldendritecscoloradoedu'
 'aa early slump' 'aa fax version' 'aa fire begin' 'aa four channel'
 'aa fri apr' 'aa however would' 'aa improve aaa']

from sklearn.feature_extraction.text import TfidfVectorizer

# Instantiate TfidfVectorizer
tfidf_vectorizer = TfidfVectorizer()

# Fit and transform the processed text data
tfidf_matrix = tfidf_vectorizer.fit_transform(df['processed_text'])

# Display the shape of the TF-IDF matrix
print("Shape of TF-IDF matrix:", tfidf_matrix.shape)

# Get feature names
feature_names = tfidf_vectorizer.get_feature_names_out()

# Display TF-IDF vectors for a few sample documents
print("\nSample TF-IDF vectors:") for i in range(3):
        print(f"\nDocument {i+1}:")
        # Get the TF-IDF vector for the current document
        doc_vector = tfidf_matrix[i]
        # Convert to dense format for easier viewing and get non-zero values
        dense_vector = doc_vector.todense()
        # Get the indices of non-zero values
        nonzero_indices = dense_vector.nonzero()[1]
        # Get the corresponding feature names and TF-IDF scores
        sample_features = [(feature_names[j], dense_vector[0, j]) for j in nonzero_indices]
        # Sort by TF-IDF score in descending order and display top 10 sample_features.sort(key=lambda x:
        x[1], reverse=True) print(sample_features[:10])

Shape of TF-IDF matrix: (11314, 91563) Sample TF-IDF

vectors:
```

```
Document 1:
[('car', 0.46512033923557805), ('tellme', 0.2556005951114642),
('bricklin', 0.24485067849500589), ('funky', 0.2264735746580837),
('enlighten', 0.20400954220171708), ('bumper', 0.19591309394211504),
('doors', 0.1713422631473172), ('sports', 0.1649879230664819),
('anyone', 0.16470777614329188), ('production', 0.16172071071579538)]

Document 2:
[('poll', 0.30862717702541836), ('experiences', 0.2593065675810059),
('clock', 0.24540607929373381), ('add', 0.20038716879710608),
('speed', 0.1961734189740292), ('attained', 0.17586642158114446),
('sinks', 0.1711121991835063), ('summarizing', 0.1711121991835063),
('detailing', 0.16722771664400635), ('oscillator',
0.1598068098896564)]

Document 3:
[('powerbook', 0.28099873979605106), ('display', 0.20138510196054912),
('anybody', 0.1817467853156778), ('heard', 0.16977944879744492),
('store', 0.15652793126217274), ('bunch', 0.14952055987893947),
('somebody', 0.14430016107902943), ('opinions', 0.14037355492477974),
('introductions', 0.1314781799171116), ('twillisecnpurdueedu',
0.1314781799171116)]

from gensim.models import Word2Vec

# Tokenize the processed text into lists of words
tokenized_text = [text.split() for text in df['processed_text']]

# Train the Word2Vec model
# window: The maximum distance between the current and predicted word within a sentence.
# min_count: Ignores all words with total frequency lower than this. # workers: Use these many
worker threads to train the model (faster training with multi-core CPUs).
# sg: Training algorithm: 1 for skip-gram, 0 for CBOW. Skip-gram is often better for finding similar
words.
word2vec_model = Word2Vec(sentences=tokenized_text, vector_size=100, window=5, min_count=5,
workers=4, sg=1)

# Demonstrate finding similar words
word_to_find_similar = "computer"
if word_to_find_similar in word2vec_model.wv: similar_words =
word2vec_model.wv.most_similar(word_to_find_similar) print(f"Words similar to
        '{word_to_find_similar}':") for word, similarity in similar_words:
                print(f"   {word}: {similarity:.4f}")
else:
        print(f"'{word_to_find_similar}' not found in the vocabulary.")
```

```
word_to_find_similar = "science"
if word_to_find_similar in word2vec_model.wv: similar_words =
word2vec_model.wv.most_similar(word_to_find_similar) print(f"\nWords similar to
        '{word_to_find_similar}':") for word, similarity in similar_words:
                print(f"   {word}: {similarity:.4f}")
else:
        print(f"'{word_to_find_similar}' not found in the vocabulary.")
```

```
Words similar to 'computer':
    programmer: 0.6821
    engineering: 0.6738
    computing: 0.6704
    communication: 0.6679
    hacker: 0.6641
    transmits: 0.6619
    highspeed: 0.6618
    architecture: 0.6590
    computers: 0.6576
    shopper: 0.6537

Words similar to 'science':
    scientific: 0.7803
    scientist: 0.7794
    fiction: 0.7675
    mathematics: 0.7343
    psychology: 0.7295
    scientists: 0.7182
    studies: 0.7149
    philosophy: 0.7112
    biology: 0.7034
    environmental: 0.6891
```

```
!pip install gensim
```

```
Requirement already satisfied: gensim in
/usr/local/lib/python3.12/dist-packages (4.3.3) Requirement already satisfied:
numpy<2.0,>=1.18.5 in
/usr/local/lib/python3.12/dist-packages (from gensim) (1.26.4) Requirement already satisfied:
scipy<1.14.0,>=1.7.0 in
/usr/local/lib/python3.12/dist-packages (from gensim) (1.13.1) Requirement already satisfied: smart-
open>=1.8.1 in
/usr/local/lib/python3.12/dist-packages (from gensim) (7.3.0.post1) Requirement already satisfied: wrapt in
/usr/local/lib/python3.12/dist-packages (from smart-open>=1.8.1-
>gensim) (1.17.3)
```

```
from gensim.models import Word2Vec
```

```
# Tokenize the processed text into lists of words
tokenized_text = [text.split() for text in df['processed_text']]

# Train the Word2Vec model
# window: The maximum distance between the current and predicted word within a sentence.
# min_count: Ignores all words with total frequency lower than this. # workers: Use these many
worker threads to train the model (faster training with multi-core CPUs).
# sg: Training algorithm: 1 for skip-gram, 0 for CBOW. Skip-gram is often better for finding similar
words.
word2vec_model = Word2Vec(sentences=tokenized_text, vector_size=100, window=5, min_count=5,
workers=4, sg=1)

# Demonstrate finding similar words
word_to_find_similar = "computer"
if word_to_find_similar in word2vec_model.wv: similar_words =
word2vec_model.wv.most_similar(word_to_find_similar) print(f"Words similar to
        '{word_to_find_similar}':") for word, similarity in similar_words:
                print(f"   {word}: {similarity:.4f}")
else:
        print(f"'{word_to_find_similar}' not found in the vocabulary.")

word_to_find_similar = "science"
if word_to_find_similar in word2vec_model.wv: similar_words =
word2vec_model.wv.most_similar(word_to_find_similar) print(f"\nWords similar to
        '{word_to_find_similar}':") for word, similarity in similar_words:
                print(f"   {word}: {similarity:.4f}")
else:
        print(f"'{word_to_find_similar}' not found in the vocabulary.")
```

```
Words similar to 'computer':
    engineering: 0.7063
    programmer: 0.6859
    highspeed: 0.6795
    electronic: 0.6704
    hacker: 0.6684
    transmits: 0.6643
    shopper: 0.6635
    conferencing: 0.6466
    bulletin: 0.6451
    computers: 0.6428

Words similar to 'science':
    scientist: 0.7834
    scientific: 0.7693
```

```
    fiction: 0.7621
    mathematics: 0.7362
    studies: 0.7187
    physics: 0.7077
    psychology: 0.7003
    biology: 0.6979
    scientists: 0.6908
    unified: 0.6850

import gensim.downloader as api

# Load pre-trained GloVe embeddings (e.g., glove-wiki-gigaword-100) # This might take some time
and requires an internet connection. try:
        glove_model = api.load("glove-wiki-gigaword-100") print("GloVe model loaded
        successfully.")

        # Demonstrate finding similar words using the loaded GloVe embeddings
        word_to_find_similar = "computer"
        if word_to_find_similar in glove_model:
                similar_words = glove_model.most_similar(word_to_find_similar) print(f"\nWords similar to
                '{word_to_find_similar}' (GloVe):") for word, similarity in similar_words:
                        print(f"   {word}: {similarity:.4f}")
        else:
                print(f"'{word_to_find_similar}' not found in the GloVe vocabulary.")

        word_to_find_similar = "science"
        if word_to_find_similar in glove_model:
                similar_words = glove_model.most_similar(word_to_find_similar) print(f"\nWords similar to
                '{word_to_find_similar}' (GloVe):") for word, similarity in similar_words:
                        print(f"   {word}: {similarity:.4f}")
        else:
                print(f"'{word_to_find_similar}' not found in the GloVe vocabulary.")

except Exception as e:
        print(f"Error loading GloVe model: {e}")
        print("Please ensure you have an internet connection to download the model.")

[==================================================]  100.0%
128.1/128.1MB downloaded
GloVe model loaded successfully.

Words similar to 'computer' (GloVe):
    computers: 0.8752
```

```
    software: 0.8373
    technology: 0.7642
    pc: 0.7366
    hardware: 0.7290
    internet: 0.7287
    desktop: 0.7234
    electronic: 0.7222
    systems: 0.7198
    computing: 0.7142

Words similar to 'science' (GloVe):
    sciences: 0.8073
    physics: 0.7915
    institute: 0.7663
    mathematics: 0.7608
    studies: 0.7590
    research: 0.7585
    biology: 0.7383
    university: 0.7315
    psychology: 0.7279
    economics: 0.7271

!pip install numpy import numpy
as np
from transformers import BertModel, BertTokenizer import torch

# Load pre-trained BERT model and tokenizer
# Using 'bert-base-uncased' as a common and relatively small model tokenizer =
BertTokenizer.from_pretrained('bert-base-uncased') model = BertModel.from_pretrained('bert-
base-uncased')

# Function to generate BERT embeddings in batches
def get_bert_embeddings(texts, model, tokenizer, batch_size=16): model.eval()  # Set the model to evaluation
        mode
        embeddings = []
        with torch.no_grad():  # Disable gradient calculation for inference
                for i in range(0, len(texts), batch_size): batch_texts =
                        texts[i:i+batch_size]
                        # Tokenize and pad the batch
                        encoded_input = tokenizer(batch_texts, padding=True, truncation=True, return_tensors='pt',
max_length=128)
                        # Move tensors to the appropriate device (CPU or GPU if
available)
                        device = torch.device("cuda" if torch.cuda.is_available()
else "cpu")
                        input_ids = encoded_input['input_ids'].to(device) attention_mask =
encoded_input['attention_mask'].to(device)
```

```
                    model.to(device)
                    # Get the embeddings
                    outputs = model(input_ids, attention_mask=attention_mask)
                    # Use the embeddings of the [CLS] token (usually the first token) as the sentence
embedding
                    batch_embeddings = outputs.last_hidden_state[:, 0, :].cpu().numpy()
                    embeddings.append(batch_embeddings) return

        np.concatenate(embeddings)

# Generate BERT embeddings for the processed text
# Using a smaller subset for demonstration due to computational cost sample_size = 1000   # Adjust as
needed
sample_texts = df['processed_text'].sample(sample_size, random_state=42).tolist()
bert_embeddings = get_bert_embeddings(sample_texts, model, tokenizer)

print("Shape of BERT embeddings:", bert_embeddings.shape) Requirement already

satisfied: numpy in
/usr/local/lib/python3.12/dist-packages  (1.26.4)

/usr/local/lib/python3.12/dist-packages/huggingface_hub/utils/
_auth.py:94: UserWarning:
The secret `HF_TOKEN` does not exist in your Colab secrets.
To authenticate with the Hugging Face Hub, create a token in your settings tab
(https://huggingface.co/settings/tokens), set it as secret in your Google Colab and restart your session.
You will be able to reuse this secret in all of your notebooks. Please note that authentication is
recommended but still optional to access public models or datasets.
    warnings.warn(

{"model_id":"6c103c5cd7964374844a605d51aa1c27","version_major":2,"vers ion_minor":0}

{"model_id":"5e2418b650f649579b053010ba13a6d7","version_major":2,"vers ion_minor":0}

{"model_id":"5b238b38b1884315876c9b34599e3166","version_major":2,"vers ion_minor":0}

{"model_id":"1d4f0a41d9694a8da40a4ba57f557852","version_major":2,"vers ion_minor":0}

{"model_id":"d55a50c1880840c99cb21e5b9df7eb53","version_major":2,"vers ion_minor":0}

Shape of BERT embeddings: (1000, 768)
```

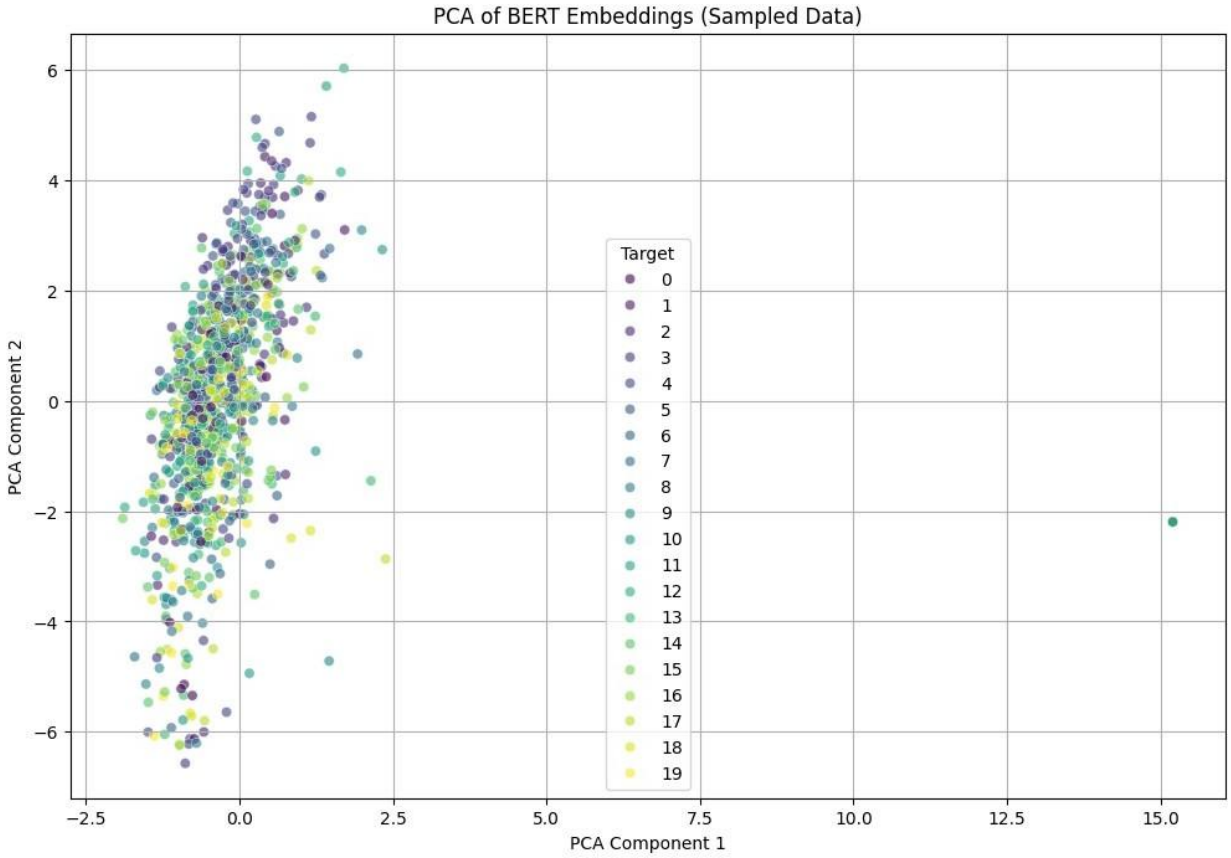
```
from sklearn.decomposition import PCA import
matplotlib.pyplot as plt import seaborn as sns

# Apply PCA for dimensionality reduction
pca = PCA(n_components=2, random_state=42) bert_pca =
pca.fit_transform(bert_embeddings)

# Create a DataFrame for plotting
pca_df = pd.DataFrame(data=bert_pca, columns=['PCA1', 'PCA2'])

# Get the corresponding target labels for the sampled texts
# Ensure the order of labels matches the order of sample_texts sample_indices =
df['processed_text'].sample(sample_size, random_state=42).index
pca_df['target'] = df.loc[sample_indices, 'target'].values

# Visualize the PCA results
plt.figure(figsize=(12, 8))
sns.scatterplot(x='PCA1', y='PCA2', hue='target', data=pca_df, palette='viridis', legend='full', alpha=0.6)
plt.title('PCA of BERT Embeddings (Sampled Data)') plt.xlabel('PCA
Component 1')
plt.ylabel('PCA Component 2') plt.legend(title='Target')
plt.grid(True)
plt.show()
```


PCA of BERT Embeddings (Sampled Data)
PCA Component 2
PCA Component 1
Target
0
1
2
3
4
5
6
7
8
9
10
11
12
13
14
15
16
17
18
19
6
4
2
0
−2
−4
−6
−2.5
0.0
2.5
5.0
7.5
10.0
12.5
15.0